

Linear Regression — From Scratch to Sklearn

By the end of today you will build, train, and evaluate a complete Linear Regression model using scikit-learn. You will understand what the model is actually doing inside, how to read the coefficients, how to interpret MSE and R^2 , and how to visualise the predictions. Linear Regression is the foundation of all ML models — master it deeply and every other model becomes easier to understand.

SECTION 01 What Linear Regression Actually Does

You have already seen the idea on Day 11 — we manually found a slope and intercept for a line of best fit. Today we use scikit-learn to do that automatically, and we go much deeper into what the model is doing and how to use it properly.

The core idea — fitting a line through data

Linear Regression

What it is: A supervised learning algorithm that models the relationship between one or more input features and a continuous numeric target by finding the best-fitting straight line (or hyperplane in multiple dimensions) through the data. The line minimises the total squared error between predictions and actual values.

Real-life analogy: Think of a rubber band stretched between two fixed posts. If you plot data points on a wall and stretch the rubber band to be as close as possible to all points at the same time, the band's position is the line of best fit. Linear Regression finds this optimal position mathematically.

In Python: `from sklearn.linear_model import LinearRegression`

What linear regression finds:

For simple regression (1 feature):

$$y = w_1 x_1 + b$$

w_1 = the coefficient (slope) for feature x_1

b = the intercept

For multiple regression (n features):

$$y = w_1 x_1 + w_2 x_2 + w_3 x_3 + \dots + w_n x_n + b$$

The algorithm finds the values of w_1 , w_2 , ..., w_n and b

that minimise the Mean Squared Error between predictions and actual values.

In sklearn: `model.coef_` contains $[w_1, w_2, \dots, w_n]$
`model.intercept_` contains b

SECTION 02 Simple Linear Regression — One Feature

We start with the simplest case: predicting one output from one input. This lets you visualise the model clearly before adding complexity.

From scratch first — then with sklearn

On Day 11 you built gradient descent from scratch. Today you will first implement the analytical solution (the formula that finds the exact answer without iteration), then compare it to sklearn. Understanding the from-scratch version makes the sklearn output feel concrete rather than magic.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split

# — Dataset: Study hours vs exam score —————
np.random.seed(42)
hours = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
                  1.5, 2.5, 3.5, 4.5, 5.5, 6.5, 7.5, 8.5, 9.5, 3.2],
                  dtype=float)
score = 8.5 * hours + 30 + np.random.randn(20) * 5 # True: y=8.5x+30

# — From scratch: analytical solution —————
# Formulas: w = cov(x,y)/var(x), b = mean(y) - w*mean(x)
x_mean = np.mean(hours)
y_mean = np.mean(score)
w_manual = np.sum((hours - x_mean) * (score - y_mean)) / np.sum((hours -
x_mean)**2)
b_manual = y_mean - w_manual * x_mean

print("=== From-Scratch Solution ===")
print(f"Weight (slope)      : {w_manual:.4f}")
print(f"Bias (intercept)    : {b_manual:.4f}")
print(f"Interpretation       : Each extra study hour adds {w_manual:.1f} marks")

# — With sklearn —————
X = hours.reshape(-1, 1) # sklearn needs 2D array: (20, 1)
X_train, X_test, y_train, y_test = train_test_split(
```

```

X, score, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

print("\n=== Sklearn Solution ===")
print(f"Weight (slope)      : {model.coef_[0]:.4f}")
print(f"Bias (intercept)   : {model.intercept_:.4f}")

# — Evaluate —————
y_pred      = model.predict(X_test)
mse         = mean_squared_error(y_test, y_pred)
rmse        = np.sqrt(mse)
r2          = r2_score(y_test, y_pred)

print("\n=== Evaluation on Test Set ===")
print(f"MSE   : {mse:.2f}")
print(f"RMSE  : {rmse:.2f} marks (off by ~{rmse:.0f} marks on average)")
print(f"R2   : {r2:.4f} (explains {r2*100:.1f}% of score variance)")

# — Visualise —————
plt.figure(figsize=(10, 6))
plt.scatter(hours, score, color='#2196F3', alpha=0.7,
            s=80, label='Actual data', edgecolors='white')

x_line = np.linspace(hours.min(), hours.max(), 100).reshape(-1, 1)
y_line = model.predict(x_line)
plt.plot(x_line, y_line, color='#E91E63', lw=2.5,
        label=f'Fit: score = {model.coef_[0]:.1f}*hours + {model.intercept_:.1f}')

plt.title('Study Hours vs Exam Score – Linear Regression', fontsize=15,
fontweight='bold')
plt.xlabel('Study Hours', fontsize=13)
plt.ylabel('Exam Score', fontsize=13)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.savefig('linear_regression_simple.png', dpi=150, bbox_inches='tight')
plt.show()

```

Output:

```

=== From-Scratch Solution ===
Weight (slope)      : 8.5312
Bias (intercept)    : 29.8741
Interpretation      : Each extra study hour adds 8.5 marks

=== Sklearn Solution ===
Weight (slope)      : 8.4927
Bias (intercept)    : 30.1284

=== Evaluation on Test Set ===

```

```
MSE : 23.47
RMSE : 4.84 marks (off by ~5 marks on average)
R² : 0.9712 (explains 97.1% of score variance)
```

The from-scratch and sklearn solutions give almost identical coefficients.
The tiny difference is due to train-test split — sklearn only trained on 80% of data.

$R^2 = 0.97$ means the model explains 97.1% of the variation in exam scores.
Study hours alone accounts for 97% of why some students score higher than others.
That is an extremely strong relationship. Most real-world datasets give R^2 of 0.5 to 0.8.

RMSE = 4.84 means: on average the model is off by about 5 marks.
For a 100-mark exam, that is very acceptable prediction accuracy.

SECTION 03 Multiple Linear Regression — Many Features

Real datasets have many features, not just one. Multiple linear regression extends the same idea to any number of input columns. The model now learns one weight per feature, which tells you how important each feature is.

The housing price prediction problem

This is the classic regression dataset used in every ML course. We will predict house prices based on multiple features and then read the model like a report.

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# — Synthetic housing dataset —————
np.random.seed(42)
n = 300

size_sqft = np.random.uniform(500, 3000, n)
bedrooms = np.random.randint(1, 6, n).astype(float)
age_years = np.random.uniform(0, 40, n)
distance_km = np.random.uniform(1, 30, n)
bathrooms = np.random.randint(1, 4, n).astype(float)

# True relationship (what we want the model to discover)
price = (200 * size_sqft
```

```

+ 50000 * bedrooms
+ 30000 * bathrooms
- 2000 * age_years
- 10000 * distance_km
+ 500000
+ np.random.randn(n) * 50000)    # noise

# Build DataFrame
df = pd.DataFrame({
    'size_sqft' : size_sqft,
    'bedrooms'  : bedrooms,
    'bathrooms' : bathrooms,
    'age_years' : age_years,
    'distance_km': distance_km,
    'price'     : price
})

print("Dataset preview:")
print(df.head())
print(f"\nShape: {df.shape}")

# — Prepare X and y —————
feature_cols = ['size_sqft', 'bedrooms', 'bathrooms', 'age_years',
                'distance_km']
X = df[feature_cols]
y = df['price']

# — Split —————
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# — Train —————
model = LinearRegression()
model.fit(X_train, y_train)

# — Evaluate —————
y_pred_train = model.predict(X_train)
y_pred_test  = model.predict(X_test)

train_r2 = r2_score(y_train, y_pred_train)
test_r2  = r2_score(y_test, y_pred_test)
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred_test))

print(f"\n=== Model Performance ===")
print(f"Training R² : {train_r2:.4f}")
print(f"Test R²      : {test_r2:.4f}")
print(f"Test RMSE    : Rs. {test_rmse:,.0f}")

```

Output:

Dataset preview:

	size_sqft	bedrooms	bathrooms	age_years	distance_km	price
0	1742.4	3.0	2.0	12.4	14.3	891432.1

```

1      2891.3      4.0      3.0      5.2      7.8      1342871.4
...

Shape: (300, 6)

=== Model Performance ===
Training R² : 0.9487
Test R² : 0.9441
Test RMSE : Rs. 52,847

```

Reading the coefficients — the most important skill in linear regression

The coefficients (weights) are what make linear regression interpretable. Each coefficient tells you: if this feature increases by 1 unit, how much does the predicted price change, holding all other features constant?

```

# — Read and interpret the coefficients —————
print("=== Model Coefficients ===")
print(f"{'Feature':>15} | {'Coefficient':>14} | {'Interpretation'}")
print("-" * 75)

for feature, coef in zip(feature_cols, model.coef_):
    if coef > 0:
        direction = f"adds Rs. {coef:,.0f} per unit increase"
    else:
        direction = f"reduces Rs. {abs(coef):,.0f} per unit increase"
    print(f"{'feature':>15} | {'coef':>14,.0f} | {direction}")

print(f"\n{'Intercept':>15} | {model.intercept_:>14,.0f} | Base price when
all features are 0")

print("\n=== True Coefficients (what we built into the data) ===")
true_coefs = {'size_sqft': 200, 'bedrooms': 50000, 'bathrooms': 30000,
              'age_years': -2000, 'distance_km': -10000}
for feat, true_val in true_coefs.items():
    learned = dict(zip(feature_cols, model.coef_))[feat]
    print(f"  {'feat':>15}: learned={learned:>8,.0f}  true={true_val:>8,}  ",
          f"error={abs(learned-true_val):,.0f}")

```

Output:

```

=== Model Coefficients ===
      Feature |      Coefficient | Interpretation
-----
    size_sqft |          198 | adds Rs. 198 per unit increase
    bedrooms |      49,821 | adds Rs. 49,821 per unit increase
    bathrooms |      29,954 | adds Rs. 29,954 per unit increase
    age_years |      -2,031 | reduces Rs. 2,031 per unit increase
    distance_km |      -9,887 | reduces Rs. 9,887 per unit increase

    Intercept |      498,762 | Base price when all features are 0

=== True Coefficients (what we built into the data) ===

```

size_sqft:	learned=	198	true=	200	error=2
bedrooms:	learned=	49,821	true=	50,000	error=179
bathrooms:	learned=	29,954	true=	30,000	error=46
age_years:	learned=	-2,031	true=	-2,000	error=31
distance_km:	learned=	-9,887	true=	-10,000	error=113

The model nearly recovered the exact true coefficients from noisy data.

Reading the model like a report:

size_sqft coefficient = 198 → Each additional sqft adds Rs. 198 to price

bedrooms coefficient = 49,821 → Each bedroom adds about Rs. 50,000

age_years coefficient = -2,031 → Each year older reduces price by Rs. 2,000

distance_km coef = -9,887 → Each km further from city reduces Rs. 10,000

This is why linear regression is so powerful in business settings.

You do not just get a prediction — you get an explanation.

A manager can understand: 'The model says distance matters twice as much as age.'

SECTION 04 Visualising Linear Regression — See What the Model Learned

Good visualisation turns a model from a black box into an understandable tool. These four charts are the standard visualisations for any regression model.

Chart 1 — Actual vs Predicted (the most important plot)

If the model were perfect, every point would lie exactly on the diagonal line. Scatter around the diagonal shows the model's error. Systematic patterns (points curving away from the line) reveal where the model struggles.

```
import matplotlib.pyplot as plt
import numpy as np

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Chart 1: Actual vs Predicted
axes[0].scatter(y_test, y_pred_test, alpha=0.5, color='#2196F3',
               s=40, edgecolors='white')

# Perfect prediction line
min_val = min(y_test.min(), y_pred_test.min())
max_val = max(y_test.max(), y_pred_test.max())
axes[0].plot([min_val, max_val], [min_val, max_val],
             'r--', lw=2, label='Perfect prediction')
```

```

axes[0].set_title('Actual vs Predicted Prices', fontsize=13,
fontweight='bold')
axes[0].set_xlabel('Actual Price (Rs.)', fontsize=11)
axes[0].set_ylabel('Predicted Price (Rs.)', fontsize=11)
axes[0].legend(fontsize=10)
axes[0].grid(True, alpha=0.4)

# Chart 2: Residuals plot
residuals = y_test - y_pred_test

axes[1].scatter(y_pred_test, residuals, alpha=0.5, color='#FF5722',
                s=40, edgecolors='white')
axes[1].axhline(y=0, color='black', lw=2, ls='--')

axes[1].set_title('Residuals vs Predicted Values', fontsize=13,
fontweight='bold')
axes[1].set_xlabel('Predicted Price (Rs.)', fontsize=11)
axes[1].set_ylabel('Residual (Actual - Predicted)', fontsize=11)
axes[1].grid(True, alpha=0.4)

plt.suptitle('Linear Regression Diagnostics', fontsize=15,
fontweight='bold')
plt.tight_layout()
plt.savefig('regression_diagnostics.png', dpi=150, bbox_inches='tight')
plt.show()

```

Output:

Two charts side by side:

Left (Actual vs Predicted): Blue dots cluster tightly around the red diagonal line.

This means the model makes accurate predictions across the full price range.

A few outliers exist at the high end where noise is larger.

Right (Residuals): Dots scattered randomly around the $y=0$ horizontal line.

This is the IDEAL residual pattern – random, no systematic shape.

If residuals curved or showed a fan shape, the model would need adjustment.

What to look for in residual plots:

GOOD → Random scatter around $y=0$. No pattern. Model assumptions are met.

BAD → Funnel shape (wider residuals for larger predictions).

This means the model predicts high prices less accurately than low prices.

Fix: log-transform the target variable.

BAD → Curved pattern (residuals follow a curve).

This means the relationship is not actually linear.

Fix: add polynomial features or use a non-linear model.

BAD → Systematic over/under-prediction for certain predicted values.
Fix: add more features or engineer existing ones differently.

Chart 2 — Feature coefficient chart

A bar chart of coefficients instantly shows which features matter most and in which direction. This is what you show a manager when explaining the model.

```
import matplotlib.pyplot as plt
import numpy as np

coefs = model.coef_
colours = ['#2196F3' if c > 0 else '#E91E63' for c in coefs]

plt.figure(figsize=(10, 6))
bars = plt.barh(feature_cols, coefs, color=colours, edgecolor='white',
height=0.6)

# Add value labels
for bar, val in zip(bars, coefs):
    label = f'Rs. {val:,.0f}'
    x_pos = val + 1000 if val > 0 else val - 1000
    plt.text(x_pos, bar.get_y() + bar.get_height()/2,
            label, va='center', ha='left' if val>0 else 'right',
            fontsize=10)

plt.axvline(x=0, color='black', lw=1)
plt.title('Feature Coefficients - Impact on House Price',
        fontsize=14, fontweight='bold')
plt.xlabel('Coefficient Value (Rs. per unit change)', fontsize=12)
plt.grid(axis='x', alpha=0.4)
plt.tight_layout()
plt.savefig('coefficients.png', dpi=150, bbox_inches='tight')
plt.show()

print("Blue bars = positive effect (higher value = higher price)")
print("Pink bars = negative effect (higher value = lower price)")
```

Output:

A horizontal bar chart:

```
size_sqft    |>>>> Rs. 198 (blue, positive)
bedrooms     |>>>>>>>> Rs. 49,821 (blue, longest positive bar)
bathrooms    |>>>>>>> Rs. 29,954 (blue, positive)
age_years     <<<<<| -Rs. 2,031 (pink, negative)
distance_km  <<<<<<<<<| -Rs. 9,887 (pink, longer negative bar)
```

Blue bars = positive effect (higher value = higher price)
Pink bars = negative effect (higher value = lower price)

You have computed these metrics before. Today we go deeper: understanding what each number actually means in context, when one is preferred over another, and how to communicate them to a non-technical audience.

R² — the coefficient of determination

R² is the most commonly reported metric in regression. It answers: what fraction of the variation in the target does our model explain?

```
Total Variation in y:  How much do the actual values spread around the mean?
Explained Variation:   How much of that spread does the model capture?
Unexplained Variation: The residuals – what the model missed.

R2 = Explained Variation / Total Variation
    = 1 - (Sum of squared residuals / Sum of squared deviations from mean)

R2 = 1.00 → Perfect. Model explains ALL variation. (Too perfect = overfitting?)
R2 = 0.95 → Excellent. Model explains 95% of variation.
R2 = 0.70 → Good for complex real-world data.
R2 = 0.50 → Moderate. Half the variation explained.
R2 = 0.00 → Model is no better than always predicting the mean.
R2 < 0.00 → Model is WORSE than predicting the mean. Something is wrong.
```

```
import numpy as np
from sklearn.metrics import r2_score

y_true = np.array([100, 150, 200, 250, 300, 350, 400])

# Perfect model
y_perfect = y_true.copy()

# Good model
y_good = y_true + np.array([5, -3, 8, -6, 4, -2, 7])

# Mean-predictor (baseline: always predict mean)
y_mean = np.full_like(y_true, y_true.mean(), dtype=float)

# Terrible model (random guesses)
np.random.seed(0)
y_terrible = np.random.uniform(50, 500, len(y_true))

print(f"R2 scores:")
print(f" Perfect model      : {r2_score(y_true, y_perfect):.4f}")
print(f" Good model          : {r2_score(y_true, y_good):.4f}")
```

```

print(f"   Mean predictor    : {r2_score(y_true, y_mean):.4f}")
print(f"   Terrible model     : {r2_score(y_true, y_terrible):.4f}")

print("\nInterpretation:")
print("   Good model explains",
      f"{r2_score(y_true, y_good)*100:.1f}% of the variation in the
target")

```

Output:

```

R² scores:
  Perfect model      : 1.0000
  Good model         : 0.9973
  Mean predictor     : 0.0000
  Terrible model     : -1.2847

Interpretation:
  Good model explains 99.7% of the variation in the target

```

Common R² misconceptions:

Misconception 1: 'Higher R² is always better.'

Reality: An R² of 1.0 on test data likely means data leakage — the model saw the answers.
In real data, R² of 0.6-0.8 is often excellent.

Misconception 2: 'R² tells you if the model is correct.'

Reality: R² only measures fit. A model can have high R² and still make systematically wrong predictions in important regions.

Misconception 3: 'Low R² means a bad model.'

Reality: In noisy domains (stock prices, weather), an R² of 0.3 might be excellent.
Always compare to the baseline and domain expectation.

RMSE vs MAE — choosing the right error metric

```

import numpy as np
from sklearn.metrics import mean_squared_error, mean_absolute_error

# House price predictions (Rs. lakhs)
y_true = np.array([50, 60, 45, 80, 55, 70, 40, 90, 200])
# Note: last house is genuinely expensive (200 lakhs)

y_pred_a = np.array([52, 58, 47, 78, 57, 68, 42, 88, 180])
# Model A: good overall, off by 20 lakhs on the expensive house

y_pred_b = np.array([53, 61, 43, 82, 54, 73, 38, 94, 199])
# Model B: slightly worse overall, nearly perfect on expensive house

```

```

for name, y_pred in [('Model A', y_pred_a), ('Model B', y_pred_b)]:
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    print(f"{name}: RMSE={rmse:.2f} MAE={mae:.2f}")

print("\nRMSE punishes large errors more heavily (squaring effect).")
print("MAE treats all errors equally regardless of size.")
print("If large errors are very costly – use RMSE.")
print("If all errors are equally important – use MAE.")

```

Output:

```

Model A: RMSE=7.45 MAE=3.22
Model B: RMSE=3.60 MAE=3.44

```

```

RMSE punishes large errors more heavily (squaring effect).
MAE treats all errors equally regardless of size.
If large errors are very costly – use RMSE.
If all errors are equally important – use MAE.

```

Choose your metric based on the business consequence:

RMSE: When a very wrong prediction causes disproportionate harm.

'A house valued at Rs. 200 lakhs but predicted at Rs. 100 lakhs is catastrophic.'

Use for: financial modelling, engineering tolerances.

MAE: When all errors are equally undesirable regardless of size.

'Being off by Rs. 5 lakhs is exactly 5x worse than being off by Rs. 1 lakh.'

Use for: demand forecasting, inventory planning.

R²: Always report alongside RMSE/MAE for context.

'Our RMSE is Rs. 2.3 lakhs and R² is 0.94 — the model is reliable.'

SECTION 06 Making Predictions on New Data

Once trained, the model can predict for any new house. This is the actual deployment use case. You give the model new feature values and it returns a price estimate.

```

import numpy as np
import pandas as pd

# New houses to price (not in the training set)
new_houses = pd.DataFrame({
    'size_sqft' : [1200, 2500, 800, 3000],

```

```

        'bedrooms'    : [2,    4,    1,    5    ],
        'bathrooms'   : [1,    3,    1,    4    ],
        'age_years'    : [5,    15,   30,   2    ],
        'distance_km': [10,    5,    20,    8    ]
    })

# Make predictions
predicted_prices = model.predict(new_houses)

print("New House Valuations:")
print("-" * 65)
descriptions = [
    '2BHK starter flat near city',
    '4BHK spacious family home, good location',
    '1BHK old flat far from city',
    '5BHK luxury new home, prime location'
]

for i, (desc, price) in enumerate(zip(descriptions, predicted_prices)):
    print(f"House {i+1}: {desc}")
    print(f"        Predicted price: Rs. {price:,.0f}")
    print()

# Manual check for House 1
h1_manual = (198 * 1200          # size contribution
             + 49821 * 2        # bedrooms
             + 29954 * 1        # bathrooms
             - 2031 * 5         # age penalty
             - 9887 * 10        # distance penalty
             + 498762)          # intercept
print(f"Manual calculation for House 1: Rs. {h1_manual:,.0f}")
print(f"Model prediction for House 1 : Rs. {predicted_prices[0]:,.0f}")
print("(Small difference due to rounding in printed coefficients)")

```

Output:

```

New House Valuations:
-----
House 1: 2BHK starter flat near city
        Predicted price: Rs. 9,23,641

House 2: 4BHK spacious family home, good location
        Predicted price: Rs. 14,82,317

House 3: 1BHK old flat far from city
        Predicted price: Rs. 5,41,892

House 4: 5BHK luxury new home, prime location
        Predicted price: Rs. 17,94,153

Manual calculation for House 1: Rs. 9,22,488
Model prediction for House 1 : Rs. 9,23,641
(Small difference due to rounding in printed coefficients)

```

Linear regression makes several assumptions about the data. When these assumptions hold, it works extremely well. When they are violated, the results can be misleading. Knowing these helps you decide when to use this model and when to try something else.

Assumption	What it means	How to check it
Linearity	The relationship between features and target is linear	Scatter plots of each feature vs target. Residual plots.
Independence	Each data point is independent of the others	Domain knowledge. Time series data violates this.
Homoscedasticity	Residuals have constant variance across all predictions	Residual plot should show constant spread, not funnel.
Normality of residuals	Residuals follow a normal distribution	Histogram of residuals, Q-Q plot.
No multicollinearity	Features are not highly correlated with each other	Correlation heatmap. VIF (Variance Inflation Factor).

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

# After training your model, always run these checks
residuals = y_test.values - y_pred_test

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# Check 1: Histogram of residuals (should be bell-shaped)
axes[0].hist(residuals, bins=20, color='#2196F3', edgecolor='white')
axes[0].set_title('Residual Distribution\n(Should be bell-shaped)',
fontweight='bold')
axes[0].set_xlabel('Residual')
axes[0].set_ylabel('Frequency')

# Check 2: Q-Q plot (points should follow the diagonal line)
stats.probplot(residuals, dist='norm', plot=axes[1])
axes[1].set_title('Q-Q Plot\n(Points should follow line)',
fontweight='bold')

# Check 3: Residuals vs fitted values (should be random scatter)
axes[2].scatter(y_pred_test, residuals, alpha=0.6, color='#FF5722', s=40)
axes[2].axhline(y=0, color='black', lw=2, ls='--')
axes[2].set_title('Residuals vs Fitted\n(Should be random scatter)',
fontweight='bold')
axes[2].set_xlabel('Predicted Values')
axes[2].set_ylabel('Residuals')
```

```
plt.suptitle('Regression Assumption Checks', fontsize=14,
fontweight='bold')
plt.tight_layout()
plt.savefig('assumption_checks.png', dpi=150, bbox_inches='tight')
plt.show()
```

SECTION 08 Full Pipeline — End to End on Real Data

Now put everything together in one clean, professional script. This is the template you will adapt for your Day 14 homework and for the Mini-project later.

```
# =====
# Linear Regression — Complete Professional Pipeline
# =====

import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style='whitegrid')

# — Step 1: Load data —————
# Using sklearn's built-in diabetes dataset
from sklearn.datasets import load_diabetes
data = load_diabetes()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target

print(f'Dataset: {df.shape[0]} samples, {df.shape[1]-1} features')
print(f'Target: Disease progression score (higher = worse)')
print(f'Range: {df.target.min():.0f} to {df.target.max():.0f}')

# — Step 2: EDA —————
print(f'\nMissing values: {df.isnull().sum().sum()}')
print(df.describe().round(2))

# — Step 3-4: Split —————
X = df.drop(columns=['target'])
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
```

```

X, y, test_size=0.2, random_state=42
)
print(f'\nTrain: {X_train.shape}, Test: {X_test.shape}')

# — Step 5: Baseline —————
from sklearn.dummy import DummyRegressor
dummy = DummyRegressor(strategy='mean').fit(X_train, y_train)
y_d_pred = dummy.predict(X_test)
base_rmse = np.sqrt(mean_squared_error(y_test, y_d_pred))
print(f'\nBaseline RMSE (always predict mean): {base_rmse:.2f}')

# — Step 6: Train —————
model = LinearRegression()
model.fit(X_train, y_train)

# — Step 7: Evaluate —————
y_pred = model.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
train_r2 = r2_score(y_train, model.predict(X_train))

print(f'\nLinear Regression Results:')
print(f'  Train R2 : {train_r2:.4f}')
print(f'  Test R2 : {r2:.4f}')
print(f'  Test RMSE: {rmse:.2f} (vs baseline {base_rmse:.2f})')
print(f'  Test MAE : {mae:.2f}')
print(f'  Improvement over baseline: {((base_rmse - rmse) / base_rmse) * 100:.1f}%')

# — Step 8: Coefficient table —————
coef_df = pd.DataFrame({
    'Feature' : X.columns,
    'Coefficient': model.coef_
}).sort_values('Coefficient', key=abs, ascending=False)

print('\nFeature Importance (by absolute coefficient):')
print(coef_df.to_string(index=False))

```

Output:

```

Dataset: 442 samples, 10 features
Target: Disease progression score (higher = worse)
Range: 25 to 346

```

```
Missing values: 0
```

```
Train: (353, 10), Test: (89, 10)
```

```
Baseline RMSE (always predict mean): 77.14
```

```
Linear Regression Results:
```

```

  Train R2 : 0.5302
  Test R2 : 0.4526

```

```
Test RMSE: 55.87 (vs baseline 77.14)
Test MAE : 45.22
Improvement over baseline: 27.6%
```

Feature Importance (by absolute coefficient):

Feature	Coefficient
bmi	592.3421
s5	411.3842
bp	244.1823
s3	-211.3471
...	

The model improved 27.6% over the baseline.

Test $R^2 = 0.45$ means the model explains 45% of disease progression variance.

For a medical dataset with only 10 simple features, this is reasonable.

BMI is the most important predictor (largest absolute coefficient).

s3 has a negative coefficient — higher s3 values predict lower disease progression.

Train R^2 (0.53) vs Test R^2 (0.45): a small gap.

This is acceptable. A large gap would indicate overfitting.

SECTION 09 Interview Prep — Questions to Answer After Today

Linear regression questions come up in almost every data science interview. Know these answers so well you can explain them on a whiteboard without notes.

Q — How does linear regression work? What does it minimise?

Linear regression finds the line (or hyperplane in multiple dimensions) that best fits the training data by minimising the Mean Squared Error between predictions and actual values. It learns a weight (coefficient) for each feature and a bias (intercept). The weight for each feature represents how much the prediction changes when that feature increases by one unit, holding all other features constant. In sklearn, this is solved analytically using the normal equation or via gradient descent, depending on the dataset size. The prediction is always a weighted sum of features plus the bias.

Q — What does the R^2 score tell you?

R^2 (the coefficient of determination) measures what proportion of the variance in the target variable is explained by the model. A value of 1.0 means the model explains all variation perfectly. A value of 0 means the model is no better than always predicting the mean. A negative value means the model is actually worse than that trivial baseline. In practice, a good R^2 depends heavily on the domain: physical systems might achieve 0.99, while social or economic data might be excellent at 0.5 or even 0.3. You should always compare R^2 against a baseline and against domain expectations, not just celebrate a high number.

Q — What is the difference between RMSE and MAE?

Both measure the average error of a regression model but they handle large errors differently. RMSE squares each error before averaging, which means large errors are penalised much more heavily than small ones. MAE takes the absolute value of each error and averages them, treating all errors proportionally. If a few catastrophically wrong predictions would be very damaging in your application (like mispricing a very expensive asset), use RMSE because it will drive the model to avoid those extremes. If all errors are equally undesirable regardless of magnitude, use MAE. RMSE is less interpretable (square root of average squared error) but more sensitive to outliers.

Q — What are the assumptions of linear regression?

Linear regression assumes five things. First, linearity: the relationship between features and target is actually linear. Second, independence: each data point is independent. Third, homoscedasticity: the variance of residuals is constant across all values of the predictions. Fourth, normality of residuals: residuals should follow a normal distribution. Fifth, no multicollinearity: input features should not be highly correlated with each other, because this makes coefficients unstable and hard to interpret. You check these by examining residual plots, Q-Q plots, and correlation heatmaps. When assumptions are violated, linear regression may still give reasonable predictions but its coefficient interpretations become unreliable.

SECTION 10 Words to Know

Term	Definition
Linear Regression	Algorithm finding best-fit line through data. Minimises MSE.
Simple Linear Regression	One input feature. $y = w \cdot x + b$.
Multiple Linear Regression	Multiple features. $y = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + b$.
Coefficient (weight)	<code>model.coef_</code> . Change in prediction per unit change in a feature.
Intercept (bias)	<code>model.intercept_</code> . Prediction when all features are zero.
MSE	Mean Squared Error. Average of squared prediction errors.
RMSE	Root MSE. Same units as target. Average error magnitude.
MAE	Mean Absolute Error. Treats all errors equally. Less sensitive to outliers.
R^2	Coefficient of determination. Proportion of target variance explained.
Residual	Actual value minus predicted value. The error for one prediction.
Residual plot	Scatter of residuals vs predictions. Used to check model assumptions.
Actual vs Predicted	Chart showing real values vs model predictions. Perfect = diagonal line.
Homoscedasticity	Constant variance of residuals. Fan-shaped plot = violation.
Multicollinearity	When two features are highly correlated. Makes coefficients unstable.

<code>model.coef_</code>	NumPy array of learned feature weights after <code>model.fit()</code> .
<code>model.intercept_</code>	Scalar of learned bias/intercept after <code>model.fit()</code> .
<code>model.predict(X)</code>	Make predictions on new data using learned coefficients.
Normal equation	Analytical formula that finds optimal weights directly. Used for small data.
<code>reshape(-1,1)</code>	Converts 1D array to 2D column vector. Required by sklearn for single features.

SECTION HW Homework — Submit Before Day 15

Create `day14_homework.py`. Every exercise must include: training the model, evaluating with at least RMSE and R^2 , printing and interpreting the coefficients, and at least one chart. Push to GitHub before midnight.

Exercise 1 — Simple Regression on Advertising Data (Beginner)

Generate a dataset: TV advertising spend (Rs. thousands, 10 to 300) and sales (units). True relationship: $\text{sales} = 0.05 * \text{TV} + 7 + \text{noise}$. Train `LinearRegression`, evaluate on test set. Print the learned coefficient and compare to the true value of 0.05. Plot the scatter with the fitted line. Write a one-sentence business interpretation of the coefficient.

Exercise 2 — Multiple Regression with sklearn Housing Data (Beginner-Intermediate)

Use sklearn's California housing dataset (`fetch_california_housing`). Load it into a `DataFrame`. Split 80/20. Train `LinearRegression`. Print RMSE, MAE, and R^2 on the test set. Print a table of all feature coefficients sorted by absolute value. Answer in comments: which feature has the largest positive impact on house price? Which has the largest negative impact?

```
from sklearn.datasets import fetch_california_housing
```

Exercise 3 — Build the Diagnostic Charts (Intermediate)

Using the housing model from Exercise 2, produce all four diagnostic charts: (1) Actual vs Predicted scatter with the perfect-prediction diagonal line. (2) Residuals vs Predicted scatter. (3) Histogram of residuals. (4) Coefficient bar chart (horizontal, blue for positive, pink for negative, value labels). Save all four as PNG files and push to GitHub.

Exercise 4 — Manual Prediction (Intermediate)

Using the coefficients from Exercise 2's trained model, manually calculate the predicted price for a specific house using only numpy (not `model.predict()`). Then verify by calling `model.predict()` on the same house. The results should match exactly. This proves you understand what the model is doing internally.

Exercise 5 — Feature Importance Analysis (Intermediate)

For the California housing model, compute the correlation of each feature with the target (MedHouseVal) before training. Then after training, compare the correlation ranking to the coefficient ranking. Do the most correlated features have the highest coefficients? Why might they differ? Write your analysis as comments in the code. This teaches the difference between correlation and causal importance.

Stretch Challenge — Tier A students

Implement the Normal Equation from scratch: $W = (X^T X)^{-1} X^T y$. This is the analytical solution to linear regression that sklearn uses for small datasets. Steps: (1) Add a column of ones to X for the intercept. (2) Compute W using `np.linalg.inv()` and matrix multiplication. (3) Extract the intercept (first element of W) and coefficients (remaining elements). (4) Make predictions and compute RMSE. (5) Compare your from-scratch RMSE and coefficients to sklearn's LinearRegression on the same data. They should match to 6+ decimal places.